

Setup APM Stack (Elasticsearch, APM, Kibana) on EC2, deploy a small containerized application that connects to said APM and throws metrics to it

1. Objective.....	2
2. Environment Overview.....	3
2.1 Architecture Diagram	4
3. Implementation	4
4. Method 1 — Manual docker run Commands	5
4.1 Elasticsearch Container Setup.....	5
4.2 Kibana Container Setup.....	7
4.3 APM Server Container Setup	9
4.4 Node.js Application & APM Agent Integration	9
4.5 Verification in Kibana	9
5. Method 2 — Docker Compose	9
5.1 Elasticsearch Container Setup.....	9
5.2 Kibana Container Setup.....	9
5.3 APM Server Container Setup	9
5.4 Python Application & APM Agent Integration	9
5.5 Verification in Kibana	9
6. Challenges Faced & Resolutions	9
7. Conclusion.....	9

1. Objective

The objective of this task was to set up the Elastic APM Stack (Elasticsearch, Kibana, and APM Server) and instrument two different sample applications with it — a Node.js Hello World application and a Python (Flask) Hello World application — using the corresponding Elastic APM agent for each (elastic-apm-node and elastic-apm[flask] respectively). Both applications send transactions, errors, and performance data to APM Server, which forwards it into Elasticsearch. Data is visualized in Kibana's Observability → APM UI, enabling monitoring and analysis of each application's performance.

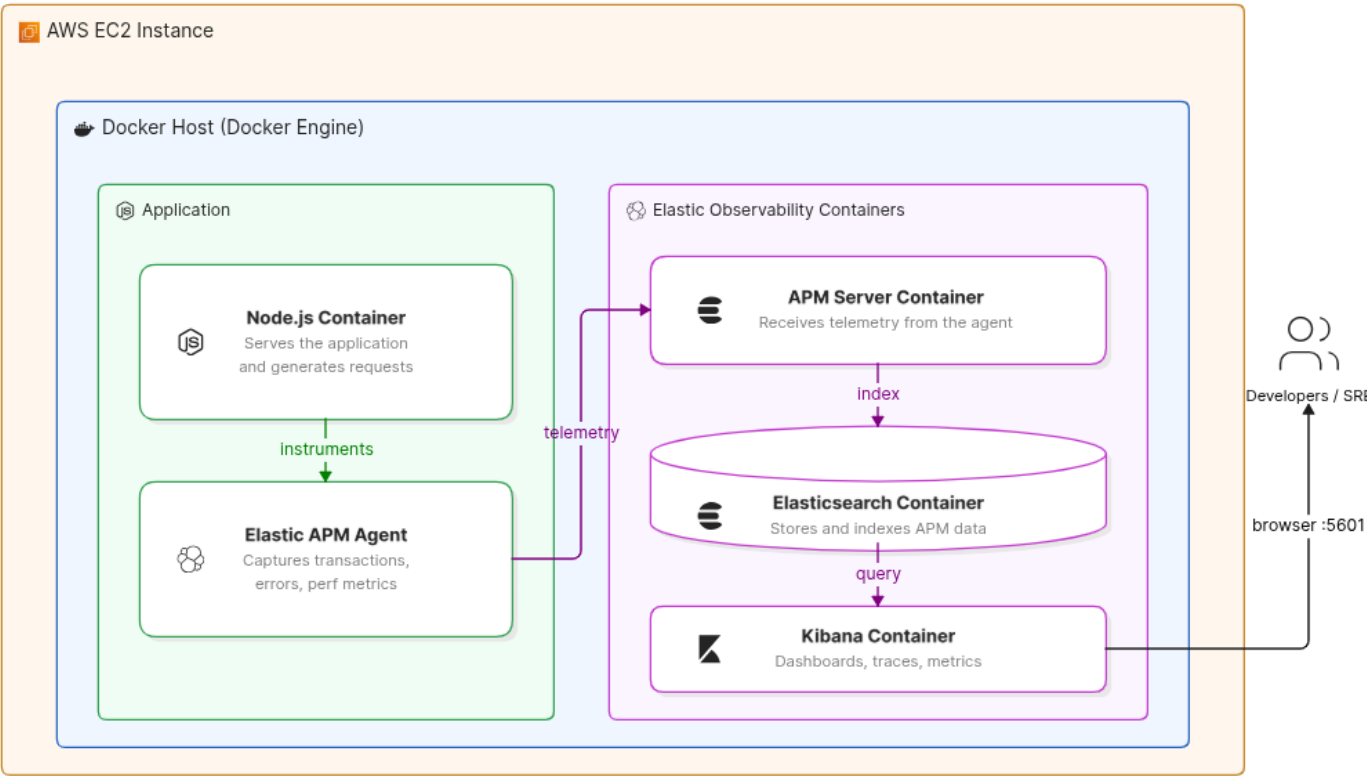
The two applications were set up using two different deployment methods, so that both a manual, step-by-step approach and a fully automated, repeatable approach are documented side by side:

- Method 1 — Manual Setup (Node.js application): Elasticsearch, Kibana, APM Server, and the Node.js Hello World application container were started individually using docker run commands, with the Docker network, environment variables, and service dependencies configured by hand at each step. This approach is useful for understanding exactly what each container needs, and for quick ad-hoc testing.
- Method 2 — Docker Compose (Python application): Elasticsearch, Kibana, APM Server, and the Python Flask Hello World application were instead defined as services in a single Docker Compose file, so the entire stack starts, in the correct dependency order, with one command. This approach is recommended for repeatable, day-to-day use, since it removes the need to remember command order, network names, or individual flags.

2. Environment Overview

Cloud Platform	AWS EC2
Containerization	Docker & Docker Compose (single docker-compose file)
Monitoring Stack	Elasticsearch, Kibana, APM Server
Sample Application	Node.js “Hello World” app (Dockerized)
Instrumentation	Elastic APM Node.js Agent

2.1 Architecture Diagram



3. Implementation

Two deployment methods were used for this stack: a manual docker run based setup (Method 1), used to instrument the Node.js Hello World application, and a Docker Compose based setup (Method 2), used to instrument the Python Flask Hello World application. Both methods deploy the same three infrastructure components — Elasticsearch, Kibana, and APM Server — plus one application container, on a shared Docker network.

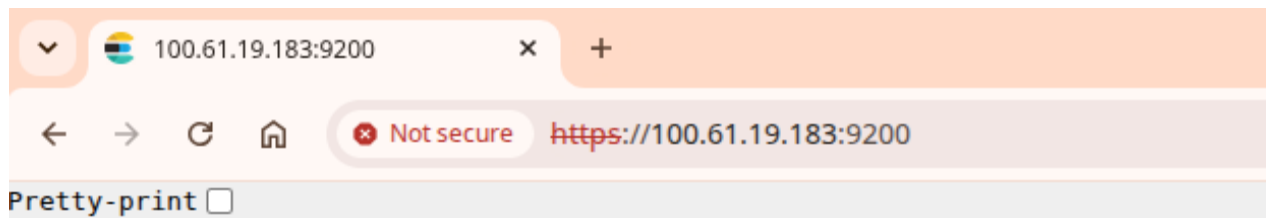
4. Method 1 — Manual docker run Commands

4.1 Elasticsearch Container Setup

Deployed the Elasticsearch container to serve as the data store for logs, metrics, and traces.

- Pulled the official Elasticsearch Docker image.
- Configured environment variables (cluster name, discovery type, memory limits, security settings).
- Started the container and verified it was reachable and healthy.

```
sudo docker run --name es01 --net elastic -p 9200:9200 -it -m 1GB  
docker.elastic.co/elasticsearch/elasticsearch:9.4.4
```

```
{
  "name" : "a278f81925ce",
  "cluster_name" : "docker-cluster",
  "cluster_uuid" : "ECXMbIt8SfGrIo92AeT_-Q",
  "version" : {
    "number" : "9.4.4",
    "build_flavor" : "default",
    "build_type" : "docker",
    "build_hash" : "77cd231096e56b56ac1d24445a9430a252622e6d",
    "build_date" : "2026-07-15T22:13:42.125968334Z",
    "build_snapshot" : false,
    "lucene_version" : "10.4.0",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

4.2 Kibana Container Setup

Deployed the Kibana container and connected it to the Elasticsearch instance for visualization.

- Pulled the official Kibana Docker image matching the Elasticsearch version.
- Set the ELASTICSEARCH_HOSTS environment variable to point to the Elasticsearch container.
- Started the container and accessed the Kibana UI in the browser.

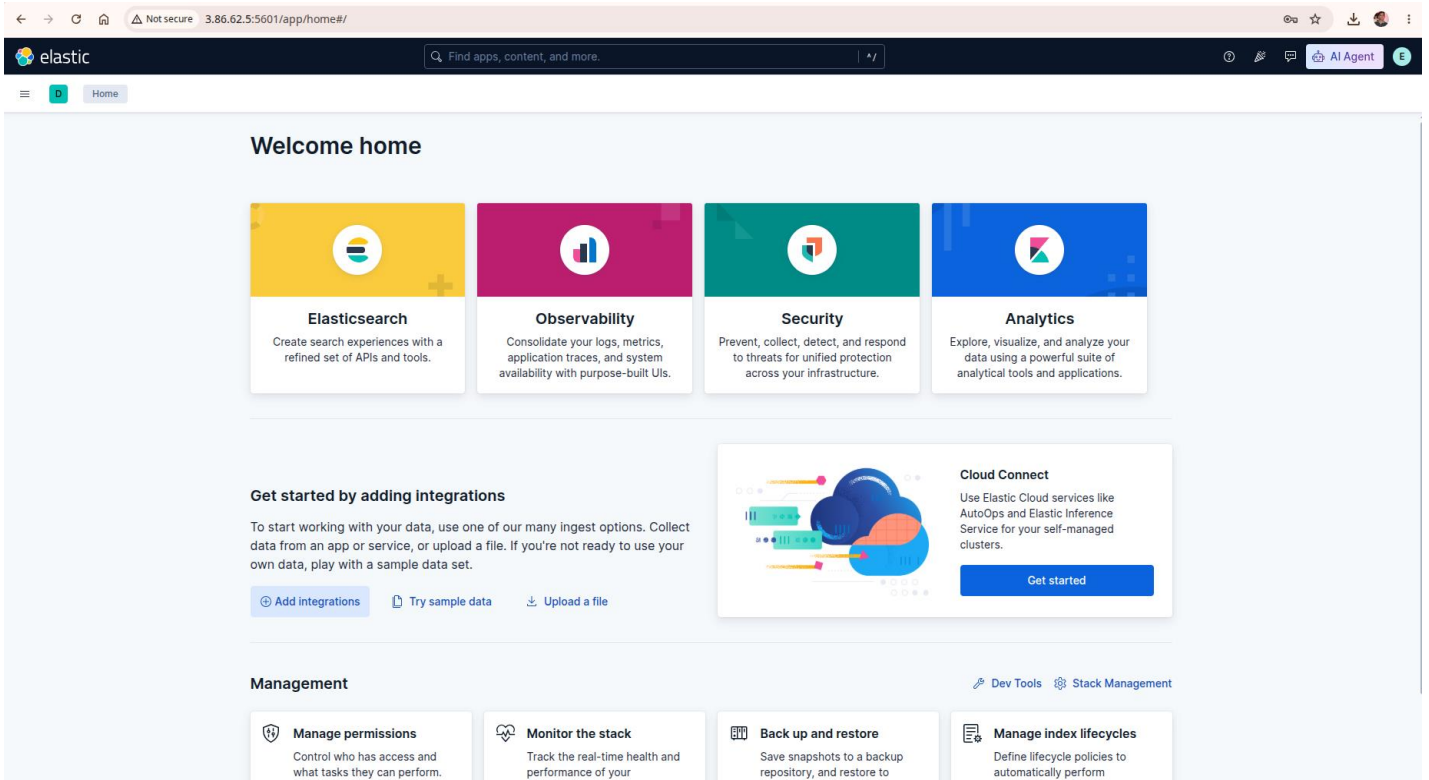
```
docker run -d \  
--name kib01 \  
--net elastic \  
-p 5601:5601 \  
docker.elastic.co/kibana/kibana:9.4.4
```


Enrollment token

Connect to <https://172.19.0.2:9200>

Configure Elastic

```
ubuntu@ip-172-31-44-204:~$ sudo docker exec -it kib01 bash
bash-5.1$ bin/kibana-verification-code
Your verification code is: 116 948
bash-5.1$
```



Once Kibana is reachable at port 5601, install the Fleet APM integration package (required before APM Server can report ready):

```
curl -X POST -u elastic:<your-elastic-password>  
"http://localhost:5601/api/fleet/epm/packages/apm" \  
-H 'kbn-xsrf: true' \  
-H 'Content-Type: application/json' \  
-d '{"force": true}'
```

4.3 APM Server Container Setup

Deployed the APM Server container to receive performance data from instrumented applications and forward it to Elasticsearch.

- Created a Dockerfile for the APM Server image.
- Created the apm-server.yml configuration file with the required settings (host/port to listen on, and the output.elasticsearch section).
- Built a custom Docker image using the Dockerfile, with the apm-server.yml file copied into the image.
- Ran a container from the built image, mapping container port 8200 to host port 8200.
- Verified the container was up and the server was listening on port 8200.

✓ Dockerfile :

```
FROM docker.elastic.co/apm/apm-server:9.4.4
COPY apm-server.yml /usr/share/apm-server/apm-server.yml
COPY http_ca.crt /usr/share/apm-server/http_ca.crt

USER root
RUN chmod 644 /usr/share/apm-server/http_ca.crt \
    && chmod go-w /usr/share/apm-server/apm-server.yml
USER 1000

EXPOSE 8200
ENTRYPOINT ["/usr/share/apm-server/apm-server", "-e"]
```

✓ Apm-server.yml

```
apm-server:
  host: "0.0.0.0:8200"
  rum:
    enabled: true
    allow_origins: ['*']

output.elasticsearch:
  hosts: ["https://es01:9200"]
  protocol: "https"
  username: "elastic"
  password: "Kunal@17198"
  ssl.certificate_authorities: ["/usr/share/apm-server/http_ca.crt"]
  ssl.verification_mode: "certificate"
  enabled: true

setup.kibana:
  host: "https://kib01:5601"

setup.template.settings:
  index:
    number_of_shards: 1
    number_of_replicas: 0

logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/apm-server
  name: apm-server.log
  keepfiles: 7
  permissions: 0644
```

4.4 Node.js Application & APM Agent Integration

Created a simple Node.js “Hello World” application, instrumented it with the Elastic APM Node.js agent, and containerized it.

- Initialized a basic Node.js Hello World application.
- Installed the elastic-apm-node package and required it as the first line of the application.
- Configured the agent with serviceName, serverUrl, and secretToken .
- Wrote a Dockerfile to containerize the application.
- Built the image and ran the container on the same Docker network as the APM stack.

Dockerfile:

```
FROM node:22-slim
```

```
WORKDIR /app
```

```
COPY package.json ./
```

```
RUN npm install
```

```
RUN npm install elastic-apm-node --save
```

```
RUN npm install express --save
```

```
COPY app.js .
```

```
EXPOSE 8080
```

```
CMD ["node","app.js"]
```

1. Build Docker image for nodejs application

```
sudo docker build -t nodejs .
```

2. Run docker container :

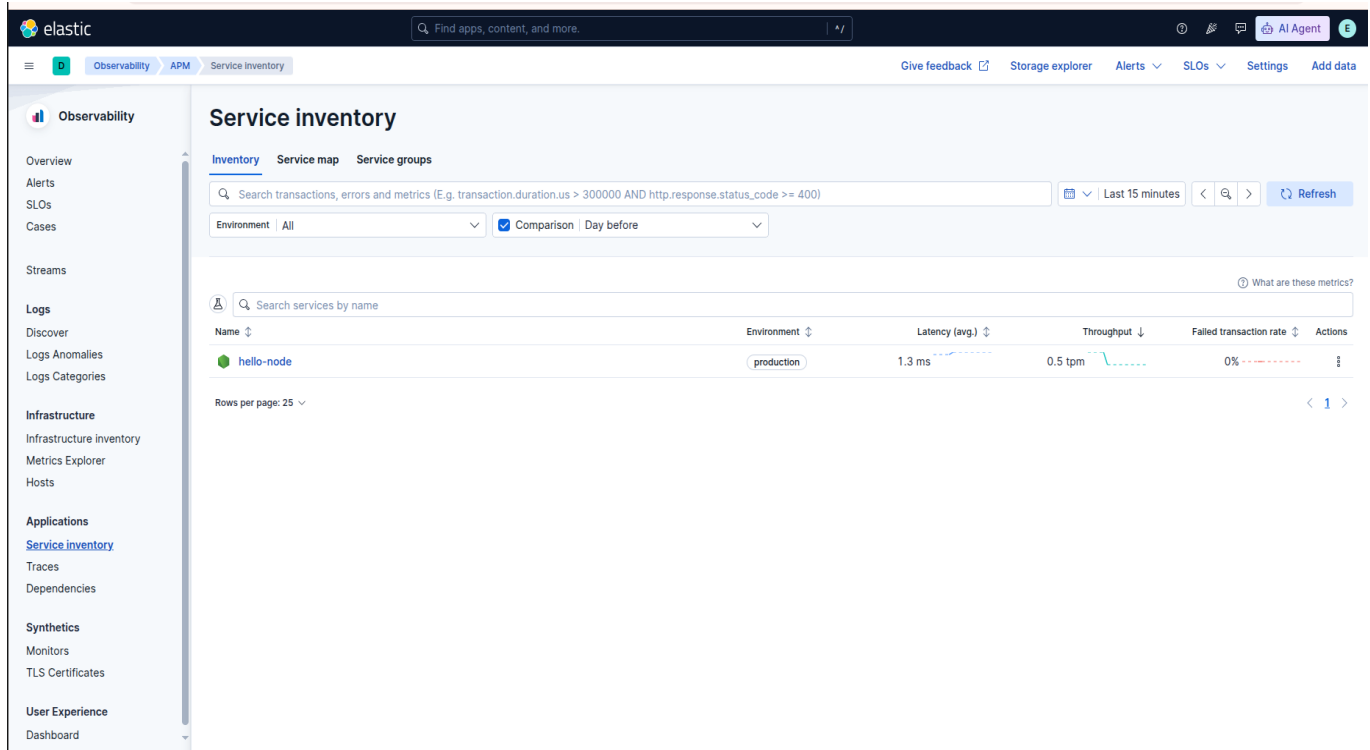
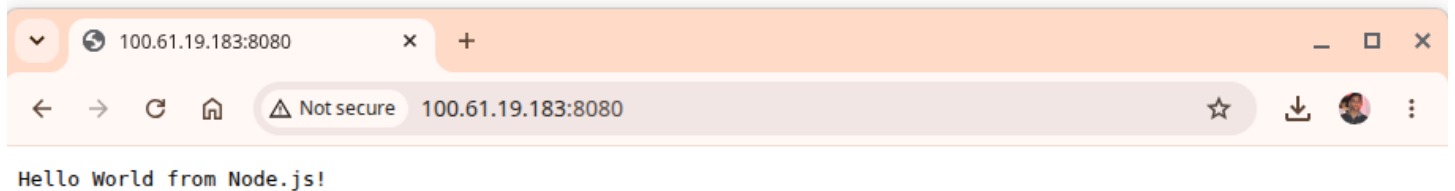
```
sudo docker run -d --name nodejs-hello --network elastic -e
```

```
ELASTIC_APM_SERVER_URL=http://apm-server11:8200 -e ELASTIC_APM_SERVICE_NAME=hello-  
node -p 8080:8080 nodejs
```

4.5 Verification in Kibana

Generated traffic on the Node.js application and confirmed that transactions and metrics appeared in Kibana.

- Sent test requests to the Hello World application endpoint.
- Opened the Kibana APM UI and confirmed the service was listed.
- Verified transactions, latency, and throughput metrics were visible.



request

Search transactions (E.g. transaction.duration.us > 300000)

Last 15 minutes

Refresh

Comparison

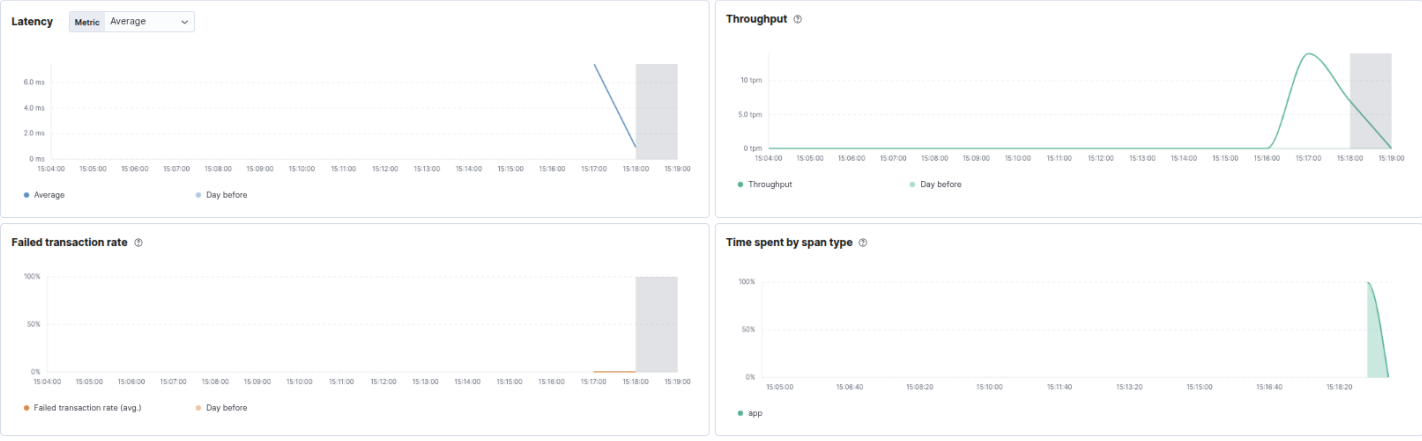
Day before

Respond quicker with SLOs

Keep your service's performance, speed, and user experience high with a Service Level Objective (SLO).

Create SLO

Hide this



5. Method 2 — Docker Compose

This method defines all five services (elasticsearch, kibana, set-kibana-password, apm-server, app) in a single Compose file, so the entire stack starts, in the correct dependency order, with one command.

```
Docker-compose:
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.14.3
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=true
      - ELASTIC_PASSWORD=Kunal@17198
      - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
    ports:
      - "9200:9200"
    volumes:
      - esdata:/usr/share/elasticsearch/data
    healthcheck:
      test: ["CMD-SHELL", "curl -s -u elastic:Kunal@17198
http://localhost:9200/_cluster/health | grep -q
\"status\": \"green\" || \"status\": \"yellow\""]
      interval: 10s
      timeout: 10s
      retries: 20
    networks:
      - apm-net
  kibana:
    image: docker.elastic.co/kibana/kibana:8.14.3
    container_name: kibana
    depends_on:
      elasticsearch:
        condition: service_healthy
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
      - ELASTICSEARCH_USERNAME=kibana_system
      - ELASTICSEARCH_PASSWORD=Kunal@17198
    ports:
      - "5601:5601"
    healthcheck:
```



```
test: ["CMD-SHELL", "curl -s http://localhost:5601/api/status | grep -q  
\"level\": \"available\""]  
interval: 10s  
timeout: 10s  
retries: 30  
networks:  
- apm-net  
apm-server:  
image: docker.elastic.co/apm/apm-server:8.14.3  
container_name: apm-server  
depends_on:  
  elasticsearch:  
    condition: service_healthy  
  kibana:  
    condition: service_healthy  
ports:  
- "8200:8200"  
command: >  
  apm-server -e  
  -E apm-server.host=0.0.0.0:8200  
  -E output.elasticsearch.hosts=["http://elasticsearch:9200"]  
  -E output.elasticsearch.username=elastic  
  -E output.elasticsearch.password=Kunal@17198  
  -E apm-server.kibana.enabled=true  
  -E apm-server.kibana.host=http://kibana:5601  
  -E apm-server.kibana.username=elastic  
  -E apm-server.kibana.password=Kunal@17198  
  -E setup.kibana.host=http://kibana:5601  
healthcheck:  
test: ["CMD-SHELL", "curl -s http://localhost:8200/ | grep -q 'ok'"]  
interval: 10s  
timeout: 10s  
retries: 20  
networks:  
- apm-net  
app:  
build: ./python-hello  
container_name: hello-world-python  
depends_on:  
- apm-server  
environment:  
- ELASTIC_APM_SERVICE_NAME=hello-world-python
```

```
- ELASTIC_APM_SERVER_URL=http://apm-server:8200
- ELASTIC_APM_ENVIRONMENT=development
- ELASTIC_APM_SECRET_TOKEN=
ports:
  - "5000:5000"
networks:
  - apm-net
volumes:
  esdata:
networks:
  apm-net:
    driver: bridge
```

5.1 Elasticsearch Container Setup

Elasticsearch was deployed as a Compose service to serve as the data store for logs, metrics, and traces, with security enabled so Kibana's Fleet feature (needed to install the APM integration package) would function.

- Used the official `docker.elastic.co/elasticsearch/elasticsearch:8.14.3` image.
- Configured single-node discovery, JVM heap limits, and `xpack.security.enabled=true`.
- Set the elastic superuser password via the `ELASTIC_PASSWORD` environment variable.
- Added a healthcheck that authenticates against the cluster health endpoint.

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.14.3
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=true
    - ELASTIC_PASSWORD=<your-elastic-password>
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
  ports:
    - "9200:9200"
  volumes:
    - esdata:/usr/share/elasticsearch/data
```

healthcheck:

```
test: ["CMD-SHELL", "curl -s -u elastic:<your-elastic-password>
```

```
http://localhost:9200/\_cluster/health | grep -q
```

```
"status":"green"\\|"status":"yellow"]
```

```
interval: 10s
```

```
timeout: 10s
```

```
retries: 20
```

networks:

```
- apm-net
```

Start only elasticsearch

```
sudo docker compose -f docker-compose up -d elasticsearch
```

Set kibana_system password to match your compose file

```
curl -X POST -u elastic:'Kunal@17198' \
```

```
-H 'Content-Type: application/json' \
```

```
http://localhost:9200/\_security/user/kibana\_system/\_password \
```

```
-d '{"password":"Kunal@17198"}'
```

```
ubuntu@ip-172-31-23-100:~/elastic-dockercompose$ sudo docker compose -f docker-compose up -d elasticsearch
[+] up 1/1
✓ Container elasticsearch Started 0.3s
ubuntu@ip-172-31-23-100:~/elastic-dockercompose$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
85a4966cb0c5	docker.elastic.co/elasticsearch/elasticsearch:8.14.3	"/bin/tini -- /usr/l..."	11 seconds ago	Up 10 seconds (health: starti

```
ng) 0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 9300/tcp elasticsearch
ubuntu@ip-172-31-23-100:~/elastic-dockercompose$ curl -X POST -u elastic:'Kunal@17198' \
-H 'Content-Type: application/json' \
http://localhost:9200/\_security/user/kibana\_system/\_password \
-d '{"password":"Kunal@17198"}'
```

Verify:

```
curl -u elastic:<your-elastic-password>
```

```
http://localhost:9200/\_cluster/health?pretty
```

5.2 Kibana Container Setup

Kibana was deployed and connected to Elasticsearch using the dedicated kibana_system service account rather than the elastic superuser.

- Used the official `docker.elastic.co/kibana/kibana:8.14.3` image.

- Pointed ELASTICSEARCH_HOSTS at the elasticsearch container over HTTP.
- Authenticated as kibana_system, with its password set by a one-shot Compose job

```
kibana:
  image: docker.elastic.co/kibana/kibana:8.14.3
  container_name: kibana
  depends_on:
    elasticsearch:
      condition: service_healthy
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    - ELASTICSEARCH_USERNAME=kibana_system
    - ELASTICSEARCH_PASSWORD=Kunal@17198
  ports:
    - "5601:5601"
  healthcheck:
    test: ["CMD-SHELL", "curl -s http://localhost:5601/api/status | grep -q
    '\"level\": \"available\"\""]
    interval: 10s
    timeout: 10s
    retries: 30
  networks:
    - apm-net
```

5.3 APM Server Container Setup

APM Server was deployed to receive performance data from the Python application and forward it to Elasticsearch, using the official image directly (no custom Dockerfile required).

- Used the official docker.elastic.co/apm/apm-server:8.14.3 image.
- Passed configuration inline via -E flags in the command, rather than a mounted apm-server.yml.
- Authenticated to Elasticsearch as the elastic user, and to Kibana as elastic for integration setup.
- Mapped container port 8200 to host port 8200 and added a healthcheck.

```
apm-server:
  image: docker.elastic.co/apm/apm-server:8.14.3
  container_name: apm-server
```

```
depends_on:
  elasticsearch:
    condition: service_healthy
  kibana:
    condition: service_healthy
ports:
  - "8200:8200"
command: >
  apm-server -e
  -E apm-server.host=0.0.0.0:8200
  -E output.elasticsearch.hosts=["http://elasticsearch:9200"]
  -E output.elasticsearch.username=elastic
  -E output.elasticsearch.password=Kunal@17198
  -E apm-server.kibana.enabled=true
  -E apm-server.kibana.host=http://kibana:5601
  -E apm-server.kibana.username=elastic
  -E apm-server.kibana.password=Kunal@17198
  -E setup.kibana.host=http://kibana:5601
healthcheck:
  test: ["CMD-SHELL", "curl -s http://localhost:8200/ | grep -q 'ok'"]
  interval: 10s
  timeout: 10s
  retries: 20
networks:
  - apm-net
```

APM Server will not report itself ready until the APM Fleet integration package is installed in Elasticsearch (it provides the traces-apm / metrics-apm.* / logs-apm.* index templates and ILM policies).

This was installed once via the Fleet API:

```
curl -X POST -u elastic:'Kunal@17198' "http://localhost:5601/api/fleet/epm/packages/apm" \
  -H 'kbn-xsrf: true' \
  -H 'Content-Type: application/json' \
  -d '{"force": true}'
```

APM Server was then restarted to re-check readiness:

```
sudo docker restart apm-server
sudo docker logs apm-server --tail 20 -f
```

5.4 Python Application & APM Agent Integration

Created a simple Python Flask Hello World application, instrumented it with the Elastic APM Flask integration, and containerized it.

- Initialized a basic Flask Hello World application with / , /health, and /error routes.
- Installed elastic-apm[flask] and configured it via app.config['ELASTIC_APM'].

- Configured SERVICE_NAME, SERVER_URL, ENVIRONMENT, and SECRET_TOKEN, all overridable by environment variables.
- Wrote a Dockerfile to containerize the application.
- Built the image and ran the container on the same Docker network (apm-net) as the APM stack, via Compose.

```
#Dockerfile:
FROM python:3.12-slim

WORKDIR /app

# Install dependencies first for better layer caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

EXPOSE 5000

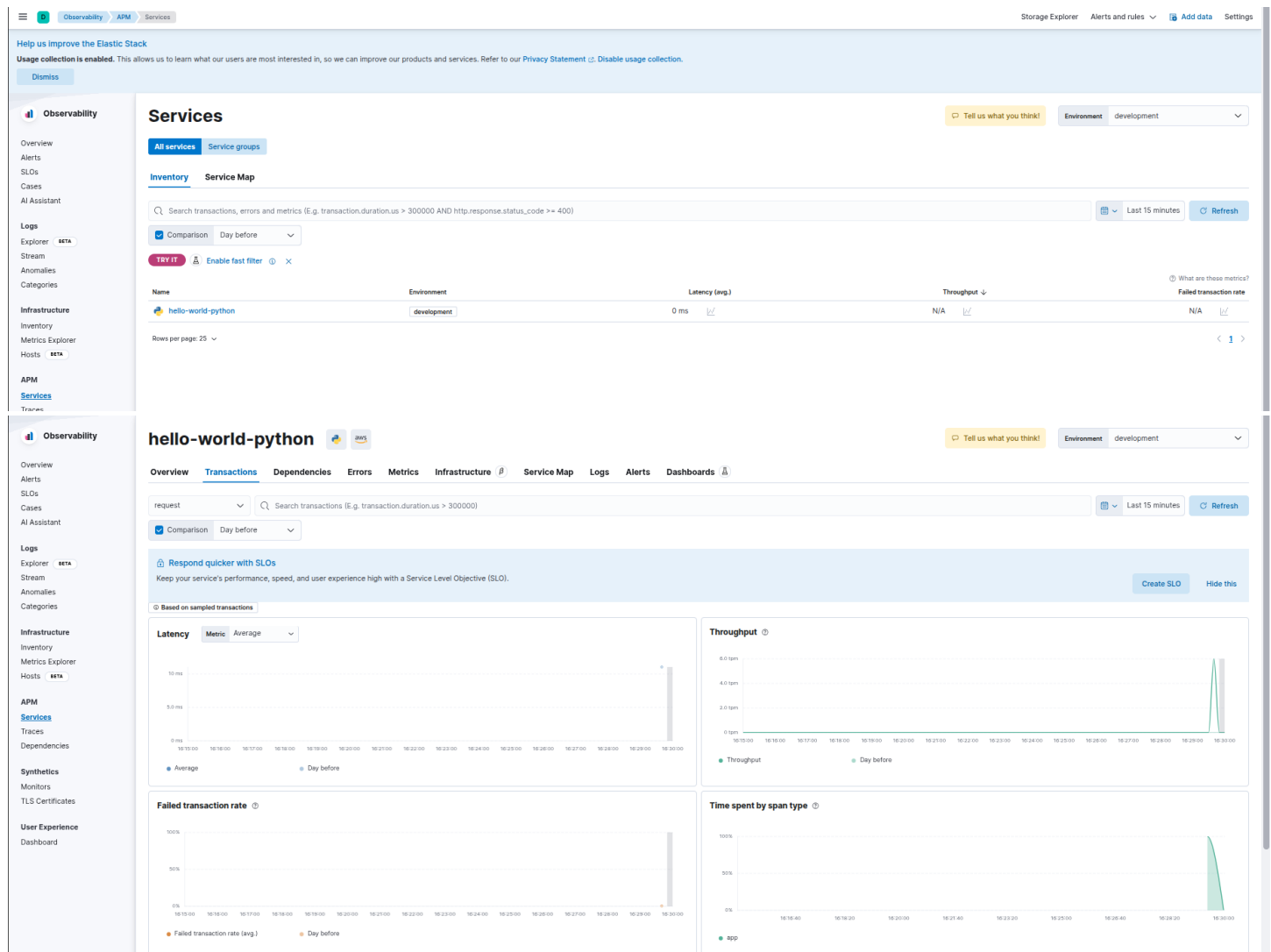
# gunicorn would be preferred for production; flask dev server used here for simplicity
CMD ["python", "app.py"]
```

```
app:
  build: ./python-hello
  container_name: hello-world-python
  depends_on:
    - apm-server
  environment:
    - ELASTIC_APM_SERVICE_NAME=hello-world-python
    - ELASTIC_APM_SERVER_URL=http://apm-server:8200
    - ELASTIC_APM_ENVIRONMENT=development
    - ELASTIC_APM_SECRET_TOKEN=
  ports:
    - "5000:5000"
  networks:
    - apm-net
```

5.5 Verification in Kibana

Generated traffic on the Python application and confirmed that transactions and errors appeared in Kibana.

- Sent test requests to the Hello World application endpoints, including the intentional error route.
- Confirmed apm-server logs showed request accepted / 202 responses with no repeated bulk indexing failures.
- Confirmed APM index data streams existed in Elasticsearch.
- Opened the Kibana APM UI (Observability → APM → Services) and confirmed the service was listed.
- Verified transactions, latency, throughput, and the captured error were visible.



6. Challenges Faced & Resolutions

During the implementation of the Elastic APM stack, a few issues were encountered. The table below summarizes the problems, their causes, and the solutions.

Issue Encountered	Resolution
APM Server could not connect to Elasticsearch.	Elasticsearch was using HTTPS , while the APM Server was trying to connect using HTTP . The configuration was updated so both services used the same protocol, and both containers were connected to the same Docker network.
CA certificate (http_ca.crt) error.	The APM Server could not trust Elasticsearch's SSL certificate. The http_ca.crt file was copied from Elasticsearch and added to the APM Server configuration to establish a secure connection.
Incorrect APM Server configuration.	Some settings in the apm-server.yml file were incorrect or missing. The configuration file was updated with the correct Elasticsearch address, credentials, and other required settings, then the container was restarted.
curl returned "publish_ready": false.	This indicated that the APM Server was running but could not communicate with Elasticsearch. After fixing the connection settings and restarting the APM Server, the status changed to "publish_ready": true .
Node.js APM agent was not sending data.	The elastic-apm-node package was not installed inside the application container. The package was added, the Docker image was rebuilt, and the APM agent was loaded before starting the Express application.

7. Conclusion

The Elastic APM Stack (Elasticsearch, Kibana, and APM Server) was successfully deployed on an AWS EC2 instance using two equivalent methods: individual docker run commands and a single Docker Compose file. A containerized Python Flask Hello World application was also deployed and configured with the Elastic APM Python Agent to send application performance data to APM Server. After resolving issues related to Docker networking and container naming, Compose build context paths, TLS/certificate trust, Fleet security requirements, service-account password provisioning, and the APM Fleet integration package, the application successfully sent transactions and error data to Elasticsearch, which was then displayed in the Kibana APM dashboard.

This task provided practical experience deploying and configuring the Elastic APM monitoring stack end-to-end via both docker run and Docker Compose, integrating a Python application with APM Server, and troubleshooting common configuration issues. The successful setup confirms that the monitoring environment is working correctly and is ready to be used for monitoring additional applications and services, helping to analyze performance, detect errors, and improve application reliability.